

ShopNow

E-Commerce Web Application

Project Documentation

Topics Covered: Tech Backbone, Payment Solutions, Security

Problem Statement

Develop a fully functional online store (e.g., bookstore or electronics) that allows product browsing, cart management, secure checkout, and dummy payment integration.

Submitted By :

Shriraj Patil	Roll No. : 50
Sumit Pawar	Roll No. : 54
Piyush Shitole	Roll No. : 63
Sumit Kore	Roll No. : 33

1. Introduction

E-commerce, also known as electronic commerce, refers to the buying and selling of goods and services over the internet. With the rapid growth of technology and the widespread use of smartphones, e-commerce has become an essential part of modern business, reshaping how consumers shop and how businesses operate. This project - ShopNow - focuses on developing a fully functional e-commerce web application that allows users to browse a product catalog, add items to a shopping cart, provide shipping details, and complete purchases through a secure online payment gateway. It provides a convenient and efficient platform for both customers and sellers.

1.1 Objectives

- Understand how online shopping platforms function end-to-end, from product listing to payment capture.
- Implement product management with data stored and served from a MySQL database via a Node.js REST API.
- Demonstrate client-side state management using JavaScript for the shopping cart.
- Integrate a real third-party payment gateway (PayPal) to handle secure online transactions.
- Understand the role of relational databases in storing customer information, product details, and transaction records.

1.2 Real-World Relevance

E-commerce systems are among the most prevalent applications of web technology today. They are used in online retail stores, ticket booking platforms, food delivery services, and digital marketplaces. This project provides hands-on experience with the practical implementation of such systems, covering the full development lifecycle from UI design to database modelling and payment processing.

2. Technologies Used

The development of ShopNow requires a combination of technologies spanning user interface design, server-side logic, data management, and payment processing. The stack follows a three-tier model: frontend, backend, and database.

Category	Technology	Purpose in Project
Frontend	HTML5	Page structure and semantic markup for all views
Frontend	CSS3	Styling, layout, animations, and responsive design

Category	Technology	Purpose in Project
Frontend	JavaScript (ES6+)	Cart logic, DOM manipulation, and Fetch API calls
Backend	Node.js	Server-side JavaScript runtime environment
Backend	Express.js	REST API routing, middleware, and request handling
Database	MySQL	Stores products, orders, order items, and customer details
Payment	PayPal JS SDK	Smart Payment Buttons for PayPal and card checkout
Config	dotenv (npm)	Manages API credentials and DB config via .env file
DB Driver	mysql2 (npm)	Node.js to MySQL connection and query execution

2.1 Frontend Technologies

- **HTML (HyperText Markup Language)** - Creates the structural skeleton of every page - the product grid, cart sidebar, checkout form, and navigation bar.
- **CSS (Cascading Style Sheets)** - Handles all visual presentation: typography, colours, card layouts, hover effects, and responsive behaviour on different screen sizes.
- **JavaScript (ES6+)** - Brings the UI to life - dynamically rendering product cards from API data, managing cart state in memory, filtering search results, and triggering PayPal payment flows.

2.2 Backend Technologies

- **Node.js** - Runs JavaScript on the server side, handling all incoming HTTP requests from the frontend in a non-blocking, event-driven manner.
- **Express.js** - A minimal web framework for Node.js that defines the three REST API routes: GET /api/products, POST /api/orders, and POST /api/orders/:id/capture.

2.3 Database

- **MySQL** - A proven relational database used to store all application data. Three tables - products, orders, and order_items - are linked via foreign keys to model the relationship between catalog items and customer purchases.

2.4 Payment Gateway

- **PayPal SDK** - Provides fully hosted, PCI-compliant Smart Payment Buttons. Users can pay via their PayPal account or a debit/credit card without any card data passing through the application server.

3. Project Architecture

ShopNow is built on a classic 3-Tier Architecture, which cleanly separates concerns into three independent layers. Each tier communicates with the next through well-defined interfaces, making the system modular, maintainable, and scalable.

Tier	Layer Name	Technology	Responsibility
1	Presentation Layer	HTML / CSS / JavaScript	Renders the UI; manages cart state; calls the REST API
2	Application Layer	Node.js + Express.js	Processes requests; enforces business logic; calls PayPal API
3	Data Layer	MySQL (ecom_db)	Persists products, orders, and order line items

3.1 Presentation Layer (Frontend)

This is the user-facing side of the application, built entirely with HTML, CSS, and JavaScript. The frontend is responsible for:

- Rendering the product catalog by fetching data from the backend API on page load.
- Managing the shopping cart in browser memory (JavaScript array).
- Collecting the customer's shipping details via a form.
- Displaying PayPal Smart Payment Buttons and initiating the payment flow.

3.2 Application Layer (Backend)

The Node.js + Express server acts as the bridge between the frontend and the database. It:

- Exposes REST API endpoints consumed by the frontend.
- Handles order creation - inserting records into MySQL with a Pending status.
- Communicates with PayPal's Orders v2 API to capture approved payments.
- Updates the order record to Completed once payment is confirmed.

3.3 Data Layer (Database)

The MySQL database ecom_db contains three tables:

Products :

Column	Type	Description
id	INT PK AUTO_INCREMENT	Unique product identifier
name	VARCHAR(255)	Product display name
price	DECIMAL(10,2)	Price in USD
image_url	TEXT	Product image URL (Unsplash)

Orders :

Column	Type	Description
id	INT PK AUTO_INCREMENT	Unique order identifier
first_name, last_name	VARCHAR(100)	Customer name
email	VARCHAR(255)	Customer email address
street_address, city, state, zip_code	VARCHAR	Shipping address
total_amount	DECIMAL(10,2)	Total order value
payment_id	VARCHAR(255)	PayPal capture ID (NULL until paid)
payment_status	VARCHAR(50)	Pending or Completed
created_at	DATETIME	Order timestamp

order_items :

Column	Type	Description
id	INT PK AUTO_INCREMENT	Unique line-item ID
order_id	INT FK → orders.id	Parent order reference
product_id	INT FK → products.id	Product purchased
quantity	INT	Units ordered
price	DECIMAL(10,2)	Unit price at time of order

3.4 Working Flow

The end-to-end request flow across all three tiers is as follows:

Step	Actor	Action
1	User / Browser	Opens the frontend; JavaScript calls GET /api/products
2	Backend (Express)	Queries MySQL products table and returns JSON array
3	User / Browser	Adds items to cart; fills shipping form; clicks PayPal button
4	Frontend → Backend	POST /api/orders - backend inserts Pending order into MySQL
5	PayPal SDK	Opens hosted modal; user authenticates and confirms payment
6	Frontend → Backend	POST /api/orders/:id/capture - backend calls PayPal capture API
7	Backend → MySQL	Updates order: payment_status = Completed, saves payment_id
8	Backend → Frontend	Returns success; frontend displays order confirmation

4. Key Features

ShopNow implements the complete set of features expected in a real-world online store. Below is a detailed overview of each feature.

4.1 Dynamic Product Catalog

Products are not hard-coded in the HTML. On page load, the frontend fetches the full product list from GET /api/products and dynamically generates product cards in the DOM. Each card displays the product image (sourced from Unsplash), name, price, and an Add to Cart button. The live catalog contains 14 products ranging from a Classic White Tee (\$29) to a Denim Jacket (\$150).

4.2 Real-Time Product Search

A search bar at the top of the shop page allows users to filter products by name as they type. Filtering is performed client-side on the already-loaded product data, providing instant results without any additional server request.

4.3 Shopping Cart with Slide-In Sidebar

Clicking Add to Cart adds the item to an in-memory JavaScript array and opens a slide-in cart sidebar from the right. The sidebar lists all cart items with price and quantity, shows a running total, and allows individual items to be removed. The navigation bar Cart button displays a live item count badge.

4.4 Checkout & Shipping Form

Clicking Checkout takes the user to the Shipping Details page. The form collects the customer's first and last name, email address, street address, city, state/province, and ZIP code. All fields must be completed before payment can be initiated.

4.5 PayPal Smart Payment Buttons

Two payment options are rendered on the checkout page by PayPal's official JavaScript SDK:

- **PayPal Button** - Opens the PayPal login modal, allowing payment via PayPal wallet, linked bank account, or saved card.
- **Debit or Credit Card Button** - Allows guest checkout without a PayPal account - card details are entered directly in PayPal's hosted, PCI-compliant form.

4.6 User Registration & Login (Scope for Extension)

The current implementation focuses on guest checkout. User accounts, login/signup, and order history are natural extensions that can be built on top of the existing orders table, where each order already records the customer's email and name.

4.7 Order Persistence & Status Tracking

Every order is written to MySQL in two stages: a Pending record is created when checkout is initiated, and it is updated to Completed - with the PayPal capture ID saved - once payment succeeds. This two-stage approach ensures abandoned carts are still recorded in the database for analytics.

5. Implementation

5.1 Frontend Implementation

Product Rendering

On DOMContentLoaded, a Fetch API call retrieves all products from the backend. The JSON response is mapped to HTML card elements which are injected into the product grid container. Each card includes the product image, name, price, and an Add to Cart button bound to a click handler.

Cart State Management

The shopping cart is maintained as a JavaScript array in memory. Adding a product pushes an object containing product id, name, price, and quantity onto the array. The renderCart() function is called on every mutation to re-draw the sidebar and refresh the cart count badge in the nav bar. Items can be removed by splicing their index from the array.

Search / Filter

The search input listens to the input event. On each keystroke, the product cards are filtered in-place: cards whose names do not contain the search string (case-insensitive) are hidden via CSS display: none; matching cards remain visible. No network request is needed.

5.2 Backend Implementation

REST API Endpoints

Method	Endpoint	Description
GET	/api/products	Returns all rows from the products table as a JSON array
POST	/api/orders	Creates a Pending order; inserts into orders + order_items; returns new order ID
POST	/api/orders/:id/capture	Calls PayPal capture API; on success sets status = Completed and saves payment_id

Order Creation Flow

When the frontend submits the checkout form, it POSTs a JSON payload containing the shipping fields and cart items array. The Express handler inserts one row into orders (with payment_status = 'Pending') and one row per cart item into order_items. The new order's id is returned to the frontend for use in the PayPal createOrder callback.

PayPal Capture Flow

On buyer approval in the PayPal modal, the frontend calls the capture endpoint. The backend uses the PayPal Orders v2 REST API to capture funds. On a successful capture response, it issues an UPDATE to set the order's payment_status to 'Completed' and records the PayPal capture ID in the payment_id column.

5.3 Database Implementation

The MySQL database ecom_db is accessed from Node.js using the mysql2 package. Connection parameters (host, port, user, password, database name) are loaded from a .env file via the dotenv package, keeping credentials out of source control. Queries use parameterised statements to prevent SQL injection.

5.4 PayPal Integration

The PayPal JavaScript SDK is loaded on the checkout page with the merchant's Sandbox client ID. The paypal.Buttons() call provides two callbacks: createOrder (which calls the backend to create the order and receives a PayPal order token) and onApprove (which triggers the capture endpoint). Switching to production requires only updating the client ID and secret in .env.

6. Implications

The implementation of an e-commerce system like ShopNow has several important implications - both the positive impact it delivers and the challenges that must be managed.

6.1 Positive Implications

- **Convenience** - Customers can shop anytime and from anywhere with an internet connection, removing the limitations of physical store hours and location.
- **Global Reach** - Businesses can reach customers worldwide without needing physical stores or local infrastructure.
- **Cost Reduction** - Eliminates overhead costs associated with physical retail such as rent, utility bills, and large support staff.
- **Faster Transactions** - Automated ordering and instant digital payment processing dramatically reduces the time between selection and purchase confirmation.
- **Better Customer Experience** - Features like real-time search, cart management, and a clean checkout flow provide a smooth, personalised shopping journey.
- **Data-Driven Decisions** - Every order and transaction is recorded in the database, enabling the business to analyse purchase patterns and improve offerings.

6.2 Challenges & Considerations

- **Security** - Handling customer data and payment information requires HTTPS, input sanitisation, and secure credential management (addressed via PayPal's PCI-compliant SDK and dotenv).
- **No Physical Interaction** - Customers cannot inspect products physically, making high-quality images and accurate descriptions critical.
- **Internet Dependency** - Both customers and the business rely on stable internet connectivity for the platform to function.
- **Scalability** - As the product catalog and order volume grow, database indexing, caching, and load balancing become necessary considerations.

7. Conclusion

ShopNow successfully demonstrates how a complete, functional e-commerce workflow can be built from scratch using fundamental, industry-standard web technologies. By deliberately avoiding heavy frontend frameworks and relying on a minimal Express backend, every layer of the application remains transparent, well-understood, and easy to extend.

The project covers the entire customer journey: product discovery, real-time search, cart management, address collection, secure payment processing via PayPal, and order persistence in a relational MySQL database. Integrating a real payment gateway with a live database elevates this beyond a typical academic exercise and into a pattern that directly mirrors real-world production systems.

The use of a 3-tier architecture ensures a clean separation of concerns, the use of MySQL demonstrates how relational data models handle real transactional data, and the PayPal integration highlights how third-party services are consumed securely using environment-based credential management.

Key Takeaways

- **REST API Design** - Designing purposeful, stateless endpoints and managing the full HTTP request/response cycle.
- **Client-Side State** - Managing UI state (cart contents) in memory using JavaScript without a framework.
- **Relational Modelling** - Normalising data across products, orders, and order_items with foreign key constraints.
- **Third-Party Integration** - Reading API documentation, handling async flows, and securely managing credentials for PayPal.
- **Full-Stack Thinking** - Understanding how each tier's responsibilities are separated and how they collaborate to deliver a feature end-to-end.

Frontend :

ShopNow

HomeShopAboutContact

Cart (0)

NEW COLLECTION 2026

Elevate Your Everyday Style

Discover our curated collection of premium essentials designed for the modern lifestyle.

Shop Now

Learn More



ShopNow

HomeShopAboutContact

Cart (0)

OUR PRODUCTS

Featured Collection

Handpicked essentials for your everyday wardrobe.

Search for products...



Classic White Tee

\$29.00

Add to Cart



Minimal Watch

\$120.00

Add to Cart



Leather Backpack

\$89.00

Add to Cart

ShopNow

[Home](#)[Shop](#)[About](#)[Contact](#)



Classic White Tee
\$29.00

Add to Cart



Minimal Watch
\$120.00

Add to Cart



Leather Backpack
\$89.00

Add to Cart



Aviator Sunglasses
\$65.00



Sport Sneakers
\$110.00



Leather Shoes
\$35.00

Your Cart

Classic White Tee
\$29.00 × 1

Minimal Watch
\$120.00 × 1

Leather Backpack
\$89.00 × 1

Total\$238.00

Checkout

ShopNow

[Home](#)[Shop](#)[About](#)[Contact](#)

Back to Shop

Shipping Details

First Name

Last Name

alex

doe

Email Address

alexdoe@alex.com

Street Address

testing temp street 01 temp apartment

City

State / Province

Zip Code

pune

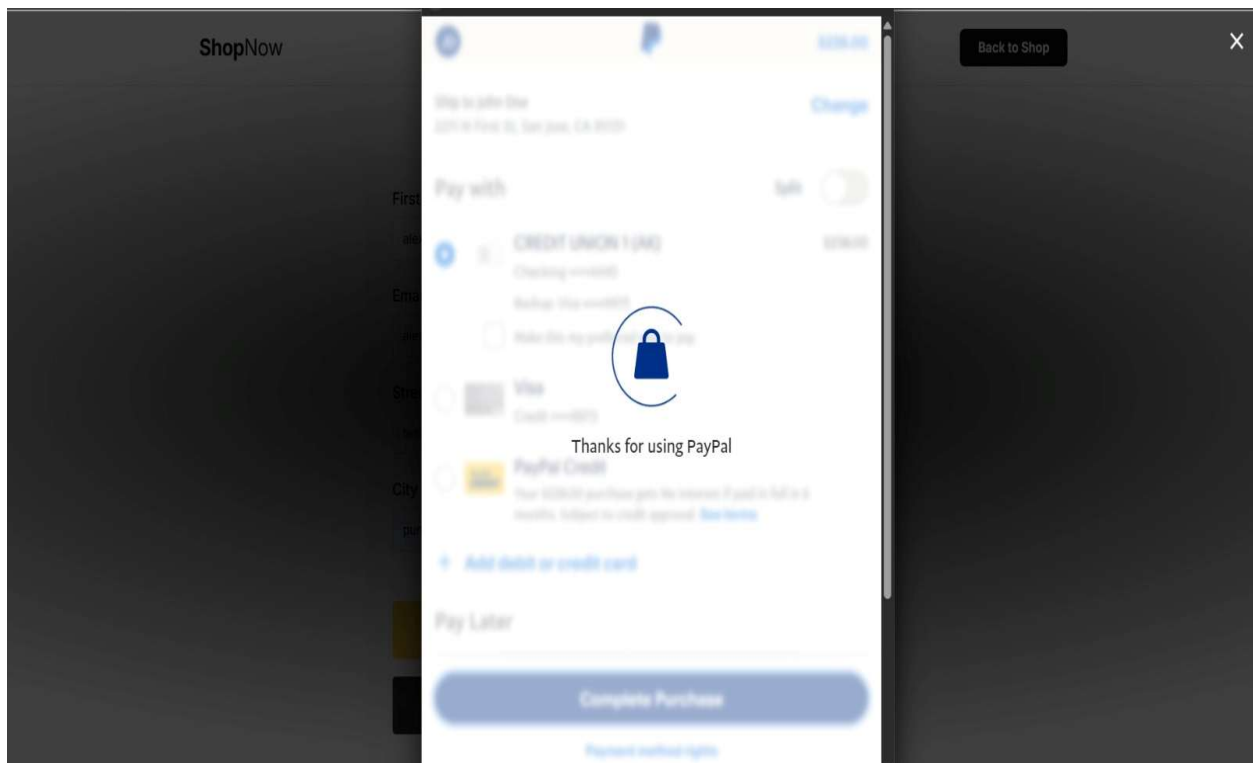
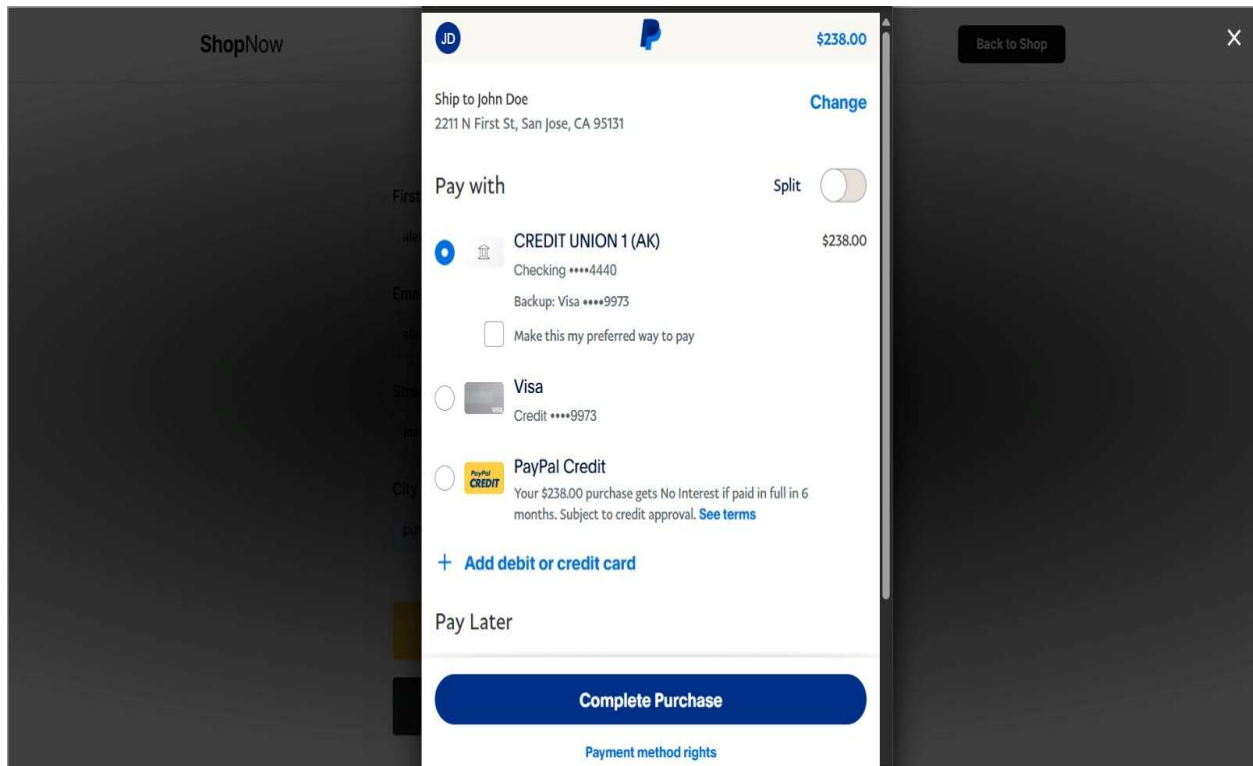
MH

10001

PayPal

Debit or Credit Card

Powered by PayPal



Backend & Database :

```
MySQL localhost:33060+ ssl SQL > use ecom_db;
Default schema set to 'ecom_db'.
Fetching global names, object names from 'ecom_db' for auto-completion... Press ^C to stop.
```

```
MySQL localhost:33060+ ssl ecom_db SQL > show tables;
```

```
+-----+
| Tables_in_ecom_db |
+-----+
| order_items        |
| orders             |
| products           |
+-----+
```

3 rows in set (0.0022 sec)

```
MySQL localhost:33060+ ssl ecom_db SQL > select * from products;
```

```
+-----+-----+-----+-----+
| id | name                | price | image_url |
+-----+-----+-----+-----+
| 1  | Classic White Tee   | 29.00 | https://images.unsplash.com/photo-1521572163474-6864f9cf17ab?w=500&q=80 |
| 2  | Minimal Watch       | 120.00 | https://images.unsplash.com/photo-1524592094714-0f0654e20314?w=500&q=80 |
| 3  | Leather Backpack    | 89.00 | https://images.unsplash.com/photo-1553062407-98eeb64c6a62?w=500&q=80 |
| 4  | Aviator Sunglasses  | 65.00 | https://images.unsplash.com/photo-1572635196237-14b3f281503f?w=500&q=80 |
| 5  | Sport Sneakers      | 110.00 | https://images.unsplash.com/photo-1542291026-7eec264c27ff?w=500&q=80 |
| 6  | Leather Shoes       | 35.00 | https://images.unsplash.com/photo-1520639888713-7851133b1ed0?w=500&q=80 |
| 7  | Denim Jacket        | 150.00 | https://images.unsplash.com/photo-1576871337622-98d48d1cf531?w=500&q=80 |
| 8  | Cotton Hoodie       | 75.00 | https://images.unsplash.com/photo-1556821840-3a63f95609a7?w=500&q=80 |
| 9  | Wool Beanie        | 25.00 | https://images.unsplash.com/photo-1576871337632-b9aef4c17ab9?w=500&q=80 |
| 10 | Running Shorts      | 45.00 | https://images.unsplash.com/photo-1667388624717-895854eea032?w=500&q=80 |
| 11 | Graphic T-Shirt     | 32.00 | https://images.unsplash.com/photo-1503342394128-c104d54dba01?w=500&q=80 |
| 12 | Winter Scarf        | 28.00 | https://images.unsplash.com/photo-1601924994987-69e26d50dc26?w=500&q=80 |
| 13 | Leather Belt        | 40.00 | https://images.unsplash.com/photo-1664286074176-5206ee5dc878?w=500&q=80 |
| 14 | Casual Sneakers     | 95.00 | https://images.unsplash.com/photo-1600269452121-4f2416e55c28?w=500&q=80 |
+-----+-----+-----+-----+
```

14 rows in set (0.0017 sec)

```
MySQL localhost:33060+ ssl ecom_db SQL > |
```

```
MySQL localhost:33060+ ssl ecom_db SQL > select * from order_items;
```

```
+-----+-----+-----+-----+-----+
| id | order_id | product_id | quantity | price |
+-----+-----+-----+-----+-----+
| 1  | 1        | 1          | 1        | 29.00 |
| 2  | 2        | 1          | 1        | 29.00 |
| 3  | 3        | 1          | 1        | 29.00 |
| 4  | 4        | 1          | 3        | 29.00 |
| 5  | 4        | 2          | 1        | 120.00 |
| 6  | 5        | 1          | 2        | 29.00 |
| 7  | 5        | 2          | 1        | 120.00 |
| 8  | 6        | 1          | 1        | 29.00 |
| 9  | 6        | 2          | 1        | 120.00 |
| 10 | 6        | 3          | 1        | 89.00 |
+-----+-----+-----+-----+-----+
```

10 rows in set (0.0014 sec)

```
MySQL localhost:33060+ ssl econ_db SQL > select * from orders;
```

id	first_name	last_name	email	street_address	city	state	zip_code	total_amount	payment_id
1	Shriraj	Patil	shriraj399@gmail.com	chopda-pune	new york	ny	10001	29.00	NULL
2	john	doe	john@test.com	jnnwfenfk	new york	ny	10001	29.00	NULL
3	john	doe	john@test.com	sjfvewjir	new york	ny	10001	29.00	6A533789NK8320
4	john	doe	john@test.com	ashfjinvjrnlekcnffnefjf	new york	ny	10001	207.00	7J1464751R0816
5	test	doe	deo@deo.com	123 sklcnlkdc	ny	ny	10001	178.00	5D094590K10870
6	alex	doe	alexdoe@alex.com	testing temp street 01 temp apartment	pune	MH	10001	238.00	3XT50010EP0846

6 rows in set (0.0023 sec)

```
localhost:5000/api/products
```

```

{
  "id": 1,
  "name": "Classic White Tee",
  "price": "29.00",
  "image_url": "https://images.unsplash.com/photo-1521572163474-6864f9cf17ab?w=500&q=80"
},
{
  "id": 2,
  "name": "Minimal Watch",
  "price": "120.00",
  "image_url": "https://images.unsplash.com/photo-1524592894714-0f0654c20314?w=500&q=80"
},
{
  "id": 3,
  "name": "Leather Backpack",
  "price": "89.00",
  "image_url": "https://images.unsplash.com/photo-1553062407-98eeb64c6a62?w=500&q=80"
},
{
  "id": 4,
  "name": "Aviator Sunglasses",
  "price": "65.00",
  "image_url": "https://images.unsplash.com/photo-1572635196237-14b3f281503f?w=500&q=80"
},
{
  "id": 5,
  "name": "Sport Sneakers",
  "price": "110.00",
  "image_url": "https://images.unsplash.com/photo-1542291026-7eec264c27ff?w=500&q=80"
},
{
  "id": 6,
  "name": "Leather Shoes",
  "price": "35.00",
  "image_url": "https://images.unsplash.com/photo-1520639888713-7851133b1ed0?w=500&q=80"
},
{
  "id": 7,
  "name": "Denim Jacket",
  "price": "150.00",
  "image_url": "https://images.unsplash.com/photo-1576871337622-98d48d1cf531?w=500&q=80"
},
{
  "id": 8,

```